

Ceranix

Backend Guide

A plain-English walkthrough of the backend plan — what we're building, why, and how it all fits together. No tech background needed.

CERANIX PLATFORM

VERSION 1.0

APRIL 2026

*Think of Ceranix like a combination of **Depop** + **WhatsApp** + a **small online mall**. People can list things they want to sell, others can browse, message the seller, make offers, and buy.*

The Big Picture

We need a reliable "engine" running behind the scenes that makes all of this work. That engine is called the **backend**.

What Is a Backend?

When you tap "Buy Now" in the app, something has to:

- Check if the item is still available
- Calculate the price + platform fee
- Record that someone bought it
- Notify the seller
- Update the listing as sold

None of that happens by magic — it runs on servers we control. The frontend (the screens you see) is already built. Our job now is to build everything that powers it.

Our Tools (Tech Stack — Simplified)

We're using **Supabase** — a Swiss Army knife for building app backends:

TOOL	WHAT IT DOES	REAL-WORLD ANALOGY
Database	Stores all data permanently	A giant, organised filing cabinet
Auth	Handles login/signup via OTP	A security guard at the door
Storage	Holds all photos	Google Photos, but for our app
Realtime	Sends live updates (chat, notifications)	A walkie-talkie between users
Edge Functions	Custom logic for complex actions	A calculator that runs our rules

The Database — What Data We're Storing

Everything is stored in **tables** — like spreadsheets, where each row is one item:

TABLE	WHAT IT STORES
profiles	Each user's name, photo, bio, location, rating
listings	Every item for sale (title, price, photos, condition)
follows	Who follows who
conversations	Chat threads between a buyer and seller about a listing
messages	Individual chat messages
offers	Offers made on listings (price, status, expiry)
orders	Completed purchases
reviews	Star ratings and comments left after a sale
notifications	Alerts (new offer, order shipped, etc.)
reports	User reports of fake/spam listings
app_config	Settings we can change (like the platform fee %)

These tables are connected. An **order** links to a **listing**, a **buyer**, and a **seller**. A **review** only exists after an **order** is completed.

The 8 Key "Workers" We're Building

Some actions are too complex for simple reads/writes. We write custom **Edge Functions** — specialist workers who handle specific jobs.

1 The Feed Ranker

Job: Decides which listings show up first in your home feed using a scoring system:

- Items from sellers you follow → **+50 points**
- Newer items → appear higher (older items slowly fade)
- Popular items (views, likes, offers) → **+20 points**
- Price-dropped items → **+10 points**
- Boosted listings → **+25 points**

Score recalculated every 30 minutes automatically.

2 The Offer Processor

Job: Handles the back-and-forth when someone makes an offer.

```
Buyer makes offer → Pending
  ↳ Seller accepts → Order automatically created
  ↳ Seller rejects → Offer closed
  ↳ Seller counters → New offer sent to buyer
  ↳ No response → Expires after 48 hours
```

3 The Buy Now Handler

Job: When someone taps "Buy Now":

1. Checks the item hasn't already been sold (at the exact same second)
2. Calculates total: item price + platform fee
3. Creates the order & marks listing as sold
4. Notifies the seller

All done as one locked action — two people can't buy the same item simultaneously.

4 The Image Upload Handler

Job: Gives sellers a secure direct upload link — faster and avoids routing through our servers.

5 The Notification Sender

Job: When something happens (new offer, message, order update), sends a real-time notification to the right person.

6 The Score Refresher AUTO · EVERY 30 MIN

Job: Updates the ranking score for every live listing so the feed stays fresh and relevant.

7 The Offer Expiry Checker AUTO · EVERY 5 MIN

Job: Scans for offers that hit their 48-hour limit and marks them as expired.

8 The Admin Tool

Job: Lets admins ban users, approve listings, change the platform fee, remove fake followers, and resolve reports.

How a Sale Works — Step by Step

1. Sara lists a dress → photos uploaded → listing goes LIVE
2. Ahmed browses the feed → ranking shows relevant listings
3. Ahmed taps listing → view counted, detail loads
4. Ahmed taps "Message Seller" → chat thread created
5. Ahmed makes offer of Rs 2,000 (listing: Rs 2,500)
→ Sara gets notification: "New offer on your Zara dress"
6. Sara counters with Rs 2,300
→ Ahmed gets notification: "Sara countered your offer"
7. Ahmed accepts
→ Order automatically created
→ Platform fee (5%): $\text{Rs } 2,300 + \text{Rs } 115 = \text{Rs } 2,415$ total
→ Both notified
8. Sara ships item, enters TCS tracking number
→ Ahmed notified: "Your order has shipped"
9. Ahmed marks as delivered
→ Sara's total sales +1
→ Ahmed can leave a review → Sara's rating updates

The Admin Panel

FEATURE	WHAT ADMINS CAN DO
Dashboard	Total users, listings, orders, and money processed (GMV)
User Management	Ban scammers, suspend rule-breakers, give verified badges
Listing Moderation	Remove fake or inappropriate listings
Reports Queue	Review user-submitted reports and resolve them
Fee Control	Change the platform's service fee percentage
Fake Follower Removal	Clean up bot-inflated follow counts
Analytics Export	Download data as a spreadsheet

Security — How We Keep Things Safe

- **Login:** OTP codes sent to phone/email (no passwords to steal)
- **Sessions:** Secure tokens that expire and auto-refresh silently
- **Data Rules:** You can only read *your own* orders — enforced at database level
- **Rate Limits:** Max 3 OTP requests/hour · Max 60 messages/minute
- **Auto-Flagging:** 3 reports on the same listing → automatically hidden and flagged for review

Real-Time Features — What Updates Instantly

FEATURE	LIVE?
Chat messages	✔ Live — appears instantly like WhatsApp
Notifications	✔ Live — badge updates immediately
Offer status changes	✔ Live
Home feed	↓ Pull-to-refresh — that's fine
Order status	✔ Live when viewing the order

Build Plan — Logical Order

We build in dependency order — you can't build the roof before the walls.

WEEK	WHAT GETS BUILT	WHY THIS ORDER
1-2	Login, profiles, photo upload	Everything else needs user accounts
2-3	Listings (create, edit, delete, publish)	Feed needs listings to show
3-4	Home feed with real ranking	Core feature of the app
4	Follow / unfollow sellers	Feed needs to know who you follow
5-6	Chat + offer system	Commerce needs conversations
6-7	Buy Now + orders + shipping	Offers need to turn into orders
7-8	Reviews, reports, notifications	Build on top of orders
8-9	Admin panel	Needs all the data to exist first
9-10	Hardening (speed, safety, stress testing)	Last before launch

Risks & How We Handle Them

RISK	IN PLAIN TERMS	HOW WE HANDLE IT
Two people buying same item	Like two people grabbing the last item off a shelf	We "lock" the item during purchase — only one goes through
Offer confusion	Seller accepts an already-expired offer	Expired offers can't be accepted — system rejects it
Too many users at once	App slows down under load	Scores pre-calculated in background; feed is just a fast lookup
Fake reviews or followers	Bots inflating seller credibility	Admin tools to detect + remove; ratio-based flagging
Changing the platform fee	Fee must apply to new orders only	Fee stored in settings table; old orders keep original fee

Summary

We're building a reliable, secure, and scalable engine that powers every tap, swipe, and transaction in the Ceranix app. The frontend already looks great — now we're wiring up the

brain behind it.

The approach keeps things simple: **one platform (Supabase), clear rules for every action, and build in logical order** so we don't redo work.

When this is done, a seller in Karachi can list a dress, a buyer in Lahore can find it, negotiate, buy it, and leave a review — all in a matter of minutes.